synkit

v 0.2.1

A toolkit to create syntactic representations in Typst

Guilherme D. Garcia

Pavillon Charles-De Koninck, 1030, Bureau DKN-3267

Département de langues, linguistique et traduction, Université Laval

Avenue des Sciences-Humaines, Québec QC, Canada

gdgarcia.ca guilhermegarcia guilherme.garcia@lli.ulaval.ca

Abstract

synkit is a Typst package for drawing syntax trees from bracket notation. It supports a wide range of features, including triangles, arrows, movement traces, multidominance, semantic annotation, cross-tree equivalence lines, and formatting options for quick adjustments. Trees can be drawn in four directions (down, up, left, right), and multiple trees can be stacked with shared arrow connections. Besides trees, the package also offers functions for numbered examples and glosses. This document serves as the official documentation for **synkit**, providing examples and explanations for all available parameters and features.

Last updated: 2026-11-06

Questions, suggestions, bugs

Any questions, comments or suggestions should be posted to the repository below (issues). All the bugs you find will help improve the package! Simply open an issue.

- `gdgarcia.ca/synkit`
- `guilhermegarcia/synkit`
- `guilhermegarcia/synkit/discussions`
- `guilhermegarcia/synkit/issues`

Note on development

syn*kit* was conceived, designed, and directed by the author. The implementation was produced with the assistance of Claude (Anthropic). All linguistic decisions, function design, and package architecture are the author's.

Version history

`0.2.1` - Bug fix: `show-refs` tags use DejaVu Sans Mono, which Typst embeds, instead of Courier New, which is missing on Linux and in web apps

`0.2.0` - `#garden()` now accepts `show-refs`. Bug fixes: words with descenders stay on the baseline in `direction: "up"` trees; `highlight` no longer shifts boxed words, and its box clears descenders; `show-refs` tags no longer cover terminal words

`0.1.1` - Bug fix: rectangular arrows now leave a trace with a visible vertical leg (the shared floor clears every arrow endpoint); unsubscripted traces in triangles (`*t*`) no longer get subscript clearance

`0.1.0` - New `#spheres()` function for Lewis-style systems of spheres (Section 5.1). Traces now use their own label (`t1` , `t2` , ...); `show-refs` displays the trace label

`0.0.41` - Bug fixes

`0.0.4` - Fixed bugs in examples

`0.0.3` - Bug fix: Greek letters in subscripts/superscripts now render correctly

`0.0.2` - Bug fixes, literal brackets now accepted as escaped characters

`0.0.1` - Initial release

Table of contents

1. Introduction 4
    1.1. Installation 4
2. Basic trees 5
3. Arrows 7
4. Multidominance 10
5. Semantics 11
    5.1. Systems of spheres 14
6. Bilingual trees 16
7. In-line movement 18
8. Numbered examples 20
9. Glosses 21
10. Extras 24
    10.1. Spacing 24
    10.2. Line breaks 26
    10.3. Formatting 27
    10.4. Direction 27
    10.5. Sizing 28
    10.6. Branch styling 29
    10.7. Font 29
11. Future work and acknowledgments 30

References 31

Appendix 32

A. Argument reference 32
B. How can I use Typst offline? 33
C. How do packages work in Typst? 33
D. Additional trees 35

1. Introduction

This vignette introduces **syn*kit*** and its functions. If you haven't used Typst yet and want to follow along, go to Typst.app to use their online editor. You may want to check their own tutorial too (I have an introductory YouTube series here). This is **not** an introduction to Typst, but see Section B and Section C in the appendix for some useful info. The GitHub repository for the package can be found at `guilhermegarcia/synkit`. Comments and suggestions are welcome, as are bug reports (open an issue in the package's repository). As with **phono*kit***, the main goals of **syn*kit*** are to MINIMIZE EFFORT and MAXIMIZE QUALITY. In a nutshell, we want more intuitive and parsimonious code than LaTeX without compromising the quality of what is created.

There are at least two fantastic packages for LaTeX to build syntax trees: `tikz-qtree` (Chiang, 2009) and `forest` (Živanović, 2017). For Typst, some options also exist: `syntree`, by Lynn (see here) and `arborly`, by Max Pearce Basman (see here). Both Typst options are indeed easier to use than anything we have for LaTeX, but they're also more limited. Thus, as of March 2026, we don't really have a syntax package that ticks enough boxes for people to consider migrating to Typst from LaTeX. This is the gap that **syn*kit*** addresses. My main goal here is to be as intuitive as possible while keeping the key features of packages used in LaTeX. Eventually, **syn*kit*** should be able to cover everything that you need in syntax (which will still be a subset of what the LaTeX packages in question can do, simply because the vast majority of people don't use 100% of what `tikz` can offer in theory, for example). I have some opinions about how an intuitive function should work, so a lot of what follows is subjective — this is also something I've applied to my other package, **phono*kit***. I realize that linguists can't simply migrate to Typst without enough coverage, and that coverage must include things like syntax trees, of course.

I am not a syntactician myself. Thus, if this package ends up being adopted by enough linguists, it would be ideal to have more people in syntax to join the project to maintain the existing functions and features.

1.1. Installation

Typst packages are always loaded the same way: using the `#import` function at the top of your `typ` document, as shown in Code 1. Replace `X.X.X` with the version you wish to import. The `*` simply states that we want to import all functions from the package in question. See the package's page on Typst's website here.

```
#import "@preview/synkit:X.X.X": *
```

Code 1: Loading a package in Typst using the official repository

Alternatively, if you want the most up-to-date version and this version is ahead of the published version, download/clone the repository here and load the package locally. This is shown in Code 2. You simply need to import the `lib.typ` file, which is present in any package.

```
#import "synkit/lib.typ": *
```

Code 2: Loading a package in Typst using a local package

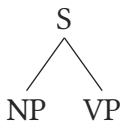
You may need to create a symlink depending on how you structure your files. See Section C for more information on Typst packages in general, as they work slightly differently from what you may be used to if you use LaTeX.

2. Basic trees

Let's begin with a simple tree, following the steps of David Chiang's excellent tutorial on `tikz-qtree` (Chiang, 2009), which you can find here. The function we'll be using is `#tree()`, and by the end of this manual, I hope that all of its parameters will be familiar. The main parameter of the function is the tree itself, which is a string, as can be seen in Code 3. The first thing to notice is that the spaces between `[]` are not a deal breaker.

```
#tree("[S [NP] [VP]]")
#tree("[S[NP][VP]]")
#tree("[ S [ NP ] [ VP ] ]")
```

Code 3: Code for Tree 1 (spaces are very flexible)

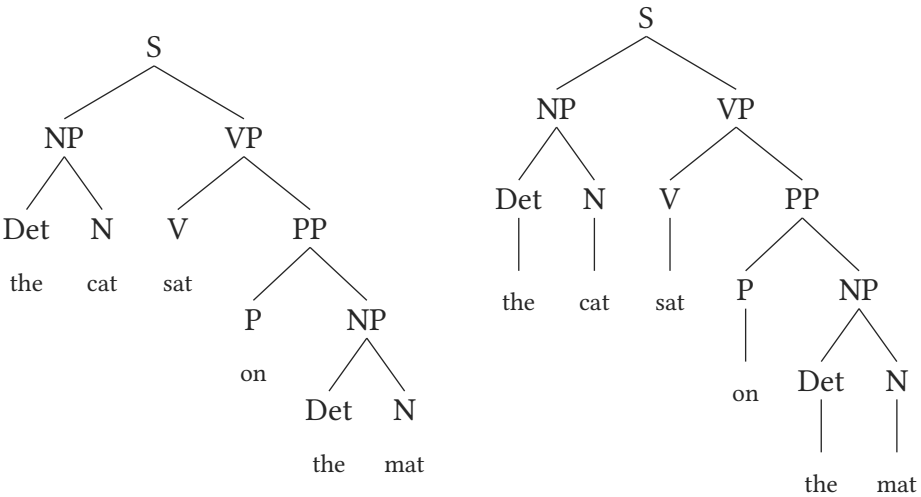


Tree 1: The basics

Next, let's complete the tree following the example in Chiang's tutorial. In Tree 2 (left), no terminal branch is drawn. This is the default behavior of `#tree()`. If you wish to produce these branches, simply add `terminal-branch: true` to the function.

```
#tree("[S [NP [Det the] [N cat]] [VP[V sat][PP[P on] [NP [Det the] [N mat]]]]]")
```

Code 4: Code for Tree 2

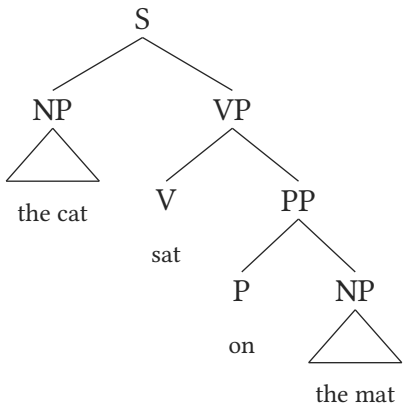


Tree 2: The basics: terminal branches deactivated by default (left)

What about adding a roof for a given XP? You can let the function decide for you whether a triangle is needed *or* you can add it yourself via the parameter `triangle`, which we'll discuss shortly. Examine Code 5: the sequence `[NP the cat]` automatically triggers a triangle because we go from phrase to content without intermediate nodes. This works because the function uses regular expressions to search for a given pattern. Time will tell how reliable this is, but thus far it's been working as it should.

```
#tree("[S [NP the cat] [VP[V sat][PP[P on] [NP the mat]]]]")
```

Code 5: Code for Tree 3

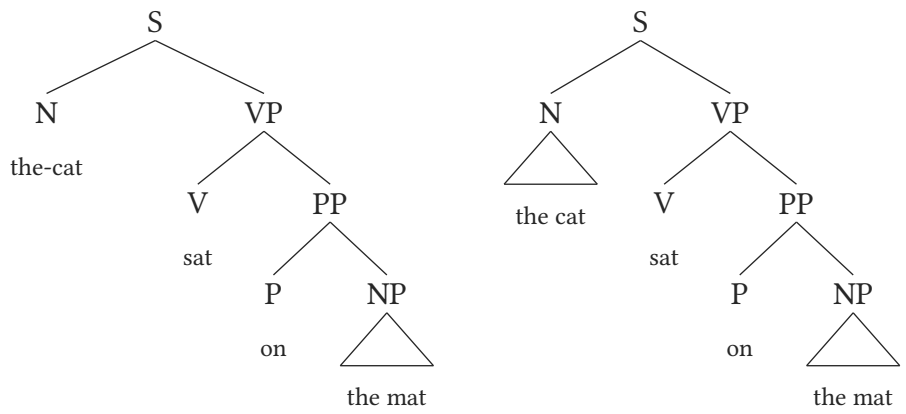


Tree 3: The basics: automatic triangles

How can we manually add triangles? If the function is successful, you will likely never need to do this. But the option is there, and it will help us understand a key element in **synkit**, namely, *labels*. To demonstrate how this works, let's pretend we want to have a triangle under the head `N` (for whatever reason), which is not a context that triggers the automatic drawing just seen. The parameter `triangle` allows you to tell the function which location in the tree should be designed with a triangle. The key here is that labels exist the moment you create a node in the tree: you never have to set labels yourself. In Tree 4, for example, we want a triangle under `N`. This is the first `N` from the top that we see in the tree (well, the only one here). Thus, we must simply state `"N1"` and the `triangle` parameter will target the correct location. This is shown in Code 6. As already mentioned, you should never need to use the `triangle` parameter, but it's there anyway. The notion of automatic labelling is essential for when we discuss arrows.

```
#tree("[S [N the cat] [VP[V sat][PP[P on] [NP the mat]]]]", triangle: ("N1",))
```

Code 6: Code for Tree 4



Tree 4: The basics: custom triangles (`N`)

Terminal node behavior: A hyphen is used in `the-cat` in Tree 4 (left) to ensure that a single atomic string appears under the terminal node. This is recommended for any multi-word expression that should be treated as one terminal, such as `couch-potato` or `pé-de-moleque` (a Brazilian candy, lit. 'boy's foot'). If you instead type `the cat`, a large space will be added between the two words, as if they were sister nodes but without branches. This is by design: `#tree()` assumes that strings in terminal position are atomic. A preterminal category can still take branching structure, which is useful for morphological or compound analyses. For example, `[N [N abc] [N def]]` renders as a noun-noun compound with internal structure, rather than as a single atomic terminal.

3. Arrows

The example below comes from Carnie (2013, p. 261). Arrows are added with the `arrows` parameter, which allows the user to specify `from` and `to` for any given arrow (this uses the automatically created labels already discussed). On top of that, the user can choose `color`, `line-width`, `dash`, and decide whether or not to curve the arrow. By default, arrows are rectangular (see gray dotted arrow in Tree 5) — multiple rectangular arrows have their heights adjusted automatically to avoid overlapping lines. This can be changed with the parameter `curved: true` (outside `arrows`). But if the user provides `bend` and `shift` inside `arrows`, `curved` will be assumed to be `true`. This is what we see in Tree 5.

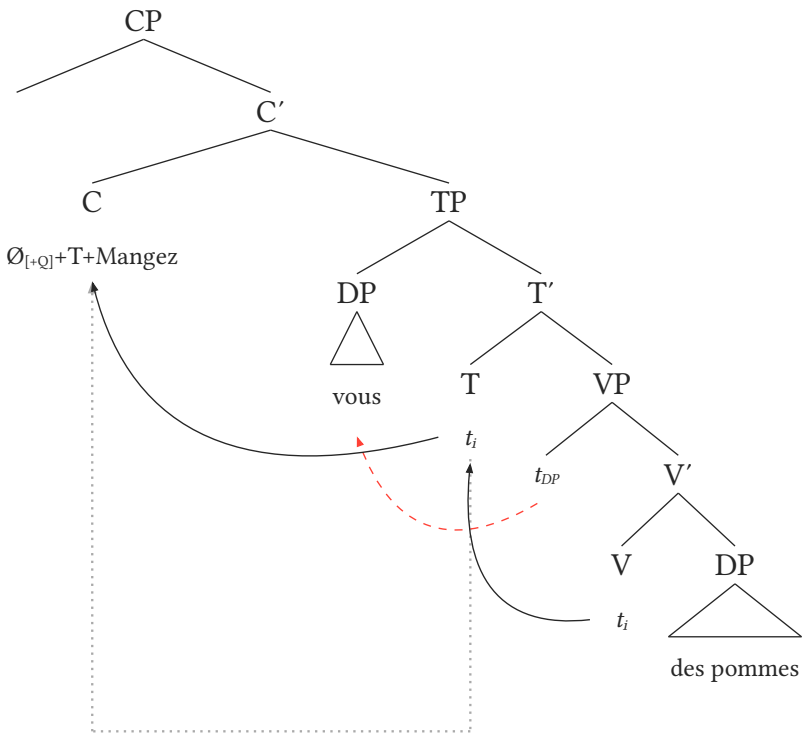
You will notice that some labels in Code 7 have a numeric suffix: `t3-198`. That suffix is optional: it lets the user define exactly *where* the arrow should depart from (or arrive at, when it's used in `to`). Imagine an invisible ellipse surrounding each node's text. The numeric suffix represents a position on that ellipse in degrees, following standard trigonometric convention: 0° is east (right), 90° is north (up), 180° is west (left), and 270° is south (down). For example, `t3-198` means "start the arrow from the 198° position around the `t3` node" — slightly south of due west. Without a degree suffix, arrows use a default exit point in the tree's growth direction (typically below the text for downward trees), as can be seen in the gray dotted arrow once again — see first line inside `arrows` in Code 7. This gives the user a higher level of control overall.

The parameters `shift` and `bend` are easy to interpret, and allow the user to adjust the curve of each arrow independently. Both parameters have default values, which are implemented when `curved: true` and

no values are provided for `bend` or `shift`. The advantage of not specifying `curved`, which is a global parameter within the function, is that both rectangular and curved arrows can coexist in the same tree.

```
#tree(
    "[ CP [] [ C' [ C Ø_{[+Q]}+T+Mangez ]
          [ TP [ DP vous ] [ T' [ T t_i ]
            [ VP [ t_{DP} ] [ V' [V t_i ] [DP des pommes] ] ] ] ] ] ]",
    arrows: (
      (from: "t1", to: "C1", dash: "dotted", color: gray, line-width: 1.70),
      (from: "t3-198", to: "T1", dash: "solid", bend: 0.2, shift: -1),
      (from: "t2-255", to: "DP1", dash: "dashed", bend: 1, shift: -0.5, color: red),
      (from: "t1-556", to: "C1", dash: "solid", bend: 1, shift: -1.5),
    ),
  )
```

Code 7: Code for Tree 5



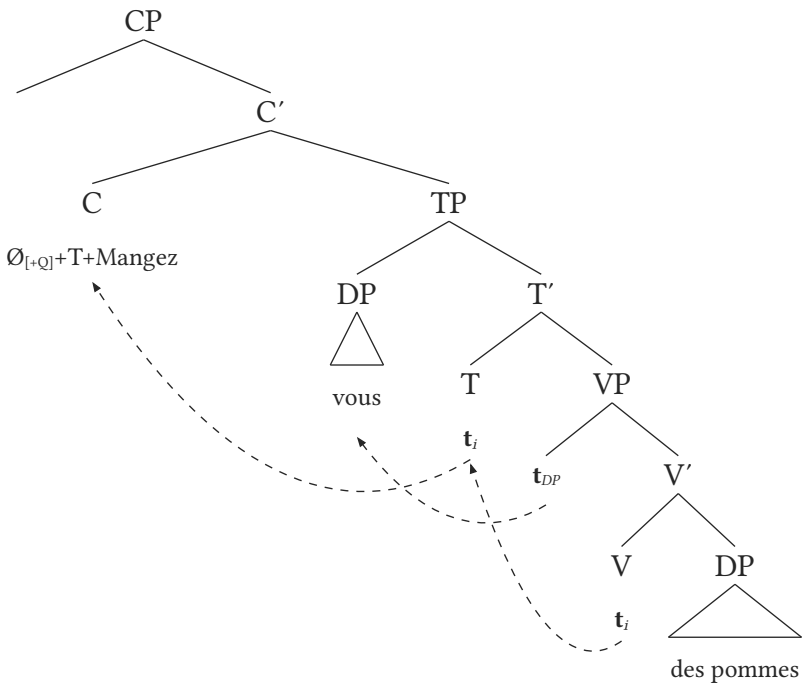
Tree 5: Multiple arrows, adapted from Carnie (2013)

One final option regarding the specific location of arrows involves two other suffixes for labels. Traces are labelled from their own content, exactly like every other node, so `t3` in Tree 5 targets the third t_i in the tree, not the V node above it, which would be targeted with `V1`. Labels are case-sensitive, so `V1` and `v1` are not interchangeable. By default, when we target a node `x`, the arrow targets its content, not the node itself. To target the node label itself instead (e.g. for phrase movement), we use a suffix: `xp1-top`. The function assumes `-down` automatically, so `xp1` is the same as `xp1-down`. On top of that, we can append degrees to customize where an arrow hits, so `xp1-down-175` would mean "target the *content* of the first XP in the tree, specifically at 175°."

In summary, arrows have a lot of defaults, so you often end up with minimal code. However, they also allow for a relatively high degree of customization through the use of additional parameters. For reference, Tree 6 uses all the default values for arrows, thus simplifying the code needed (Code 8) — `curved` is set to `true` here because multiple movements look better with curved arrows.

```
#tree(
  "[ CP [] [ C' [ C Ø_{[+Q]}+T+Mangez ] [ TP [ DP vous ] [ T' [ T *t*_i ] [ VP
[ *t*_{DP} ]
   [ V' [V *t*_i ] [DP des pommes] ] ]  ] ] ] ]",
  arrows: (
    (from: "t3", to: "T1"),
    (from: "t2", to: "DP1"),
    (from: "t1", to: "C1"),
  ),
  curved: true,
)
```

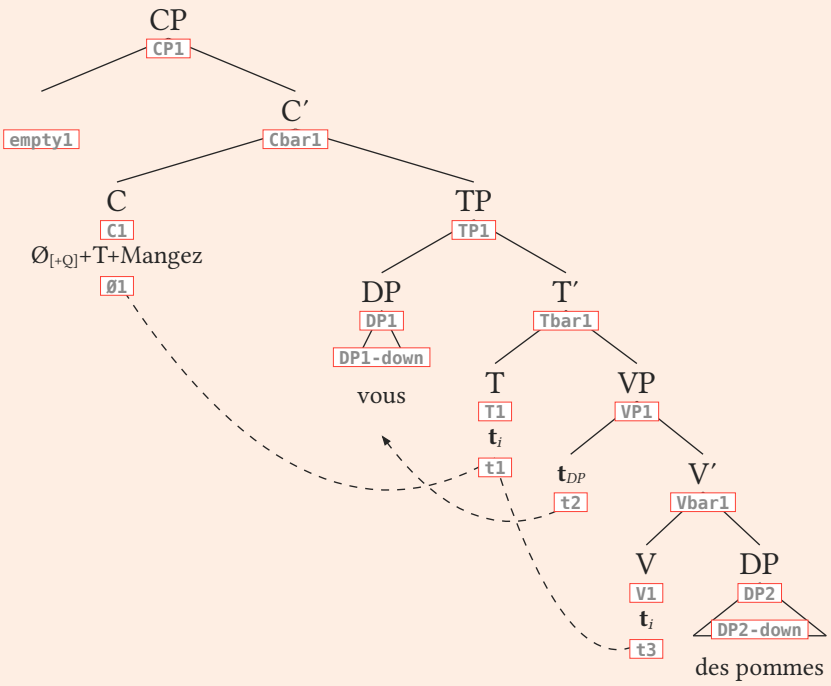
Code 8: Code for Tree 6



Tree 6: Multiple arrows, adapted from Carnie (2013)

Help with labels? Automatic labels are designed to be easy to use, and code becomes more minimal. However, complicated trees will have many more labels, and it may be annoying to target the right NP if a tree has seven of them. Furthermore, as nodes become more complex, labels also become less obvious, as we will see later in this manual. For that reason, as of version `0.0.3`, the `#tree()` function provides a helper argument: `show-refs` (defaults to `false`). When set to `true`, this will print all labels on top of all nodes for the user (this is inspired by what **phonokit** already uses in multi-tier representations). Once

all arrows are added and labels are no longer needed, the user can easily turn them off. Tree 6 is repeated below with `show-refs: true`.



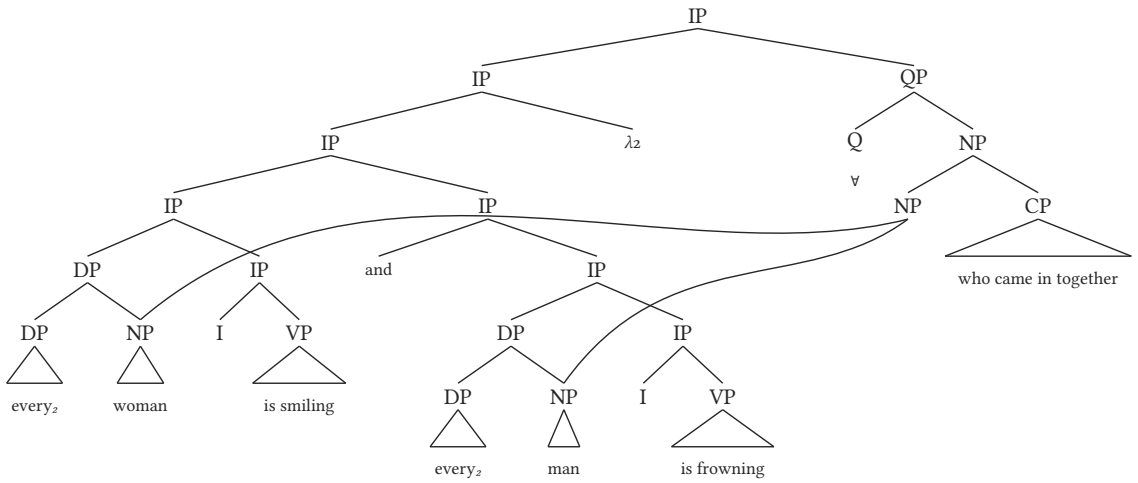
Tree 7: Displaying labels temporarily

4. Multidominance

Another important parameter in the function is `dominance`, which allows for more complex trees where multiple nodes are connected with curved branches. The example in Tree 8 comes from Fox & Johnson (2016, p. 7).

```
#tree(
  "[IP [IP [IP [IP [DP [DP every_2] [NP woman] ] [IP [I] [VP is smiling] ] ] [IP [and] [IP
[DP [DP every_2] [NP man] ] [IP [I] [VP is frowning] ] ] ] ] [\\lambda2] ] [QP [Q \
\forall] [NP [NP] [CP who came in together] ] ] ]",
  dominance: (
    (from: "NP4", to: "NP1", ctrl: (-4, 4.2)),
    (from: "NP4", to: "NP2", ctrl: (-3, 3.5)),
  ),
  scale: 0.7, // easily scale down tree to fit the document
  line-width: 1.5, // useful when tree is scaled down
  spread-local: ( // adjusts width locally, based on labels
    ("IP3", 0.6), // adjust the spacing for the third IP from the top (sister to lambda)
  ),
)
```

Code 9: Code for Tree 8



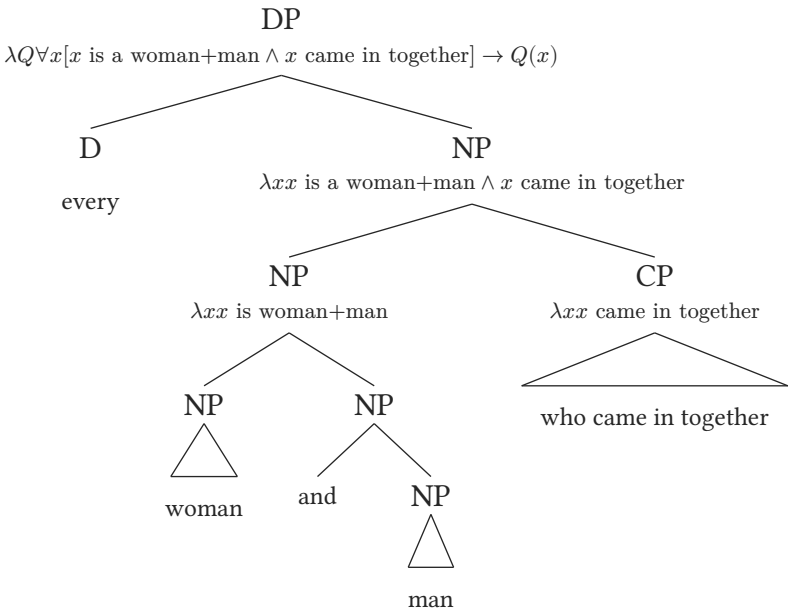
Tree 8: Multidominance

5. Semantics

A parameter is dedicated to additional content between a given node and its branches: `annotation`. This allows the user to add semantic derivations, for example. The parameter in question, shown in Code 10, uses **content blocks**, which means you can mix strings and math mode together with ease (unlike the `input` parameter for tree specification in the function). Math mode in Typst is more minimalistic than math mode in LaTeX (e.g., no backslashes are needed). Finally, `annotation-size` allows the user to set the font size for the annotations. The example in Tree 9 also comes from Fox & Johnson (2016, p. 7).

```
#tree(
  "[DP [D every] [NP [NP [NP woman] [NP [and] [NP man] ] ] [CP who came in together] ] ]",
  annotation: (
    ("DP1", [$lambda Q forall x [x "is a woman+man" and x "came in together"] arrow Q(x)$]),
    ("NP1", [$lambda x x "is a woman+man" and x "came in together"$]),
    ("NP2", [$lambda x x "is woman+man"$]),
    ("CP1", [$lambda x x "came in together"$]),
  ),
)
```

Code 10: Code for Tree 9

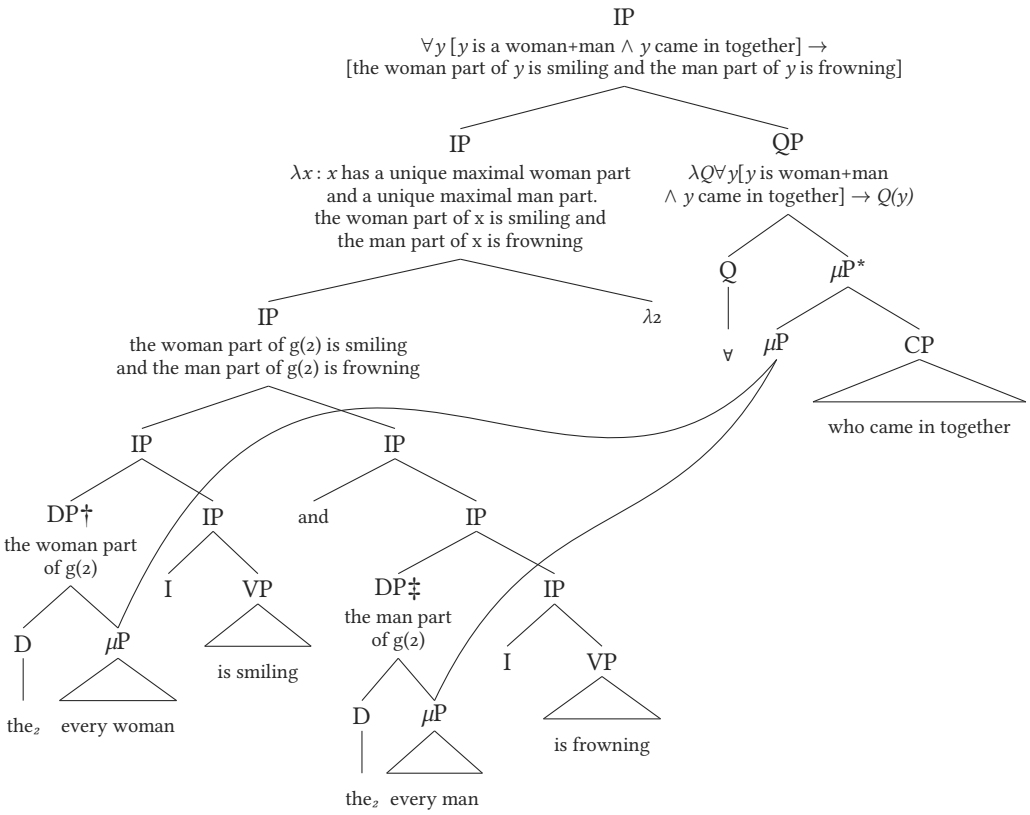


Tree 9: Semantic annotation with `annotation`

A final example includes longer annotation, from Fox & Johnson (2016, p. 13). Here, annotation involves multiple lines, so line breaks are manually added with `\`. You will notice that the code for the tree itself is relatively concise, but each annotation has its own entry, so Code 11 becomes longer. As you can see, any symbol can be directly added to a tree (e.g., ‡) — the symbol is then inherited by default by the label created for a given node, as can be seen in the annotation under DP‡ in Code 11.

```
  #tree(
  "[IP [IP [IP [IP [DP† [D the_2] [\\muP every woman] ] [IP [I] [VP is smiling] ] ]
      [IP [and] [IP [DP‡ [D the_2] [\\muP every man] ] [IP [I] [VP is frowning] ] ] ] ]
      [\\lambda2] ] [QP [Q \\forall ] [\\muP\\* [\\muP] [CP who came in together] ] ] ]",
  annotation: (
    (
      "IP1",
      [$forall$_y_ [_y_ is a woman+man $and$ _y_ came in together] $arrow$ \
        [the woman part of _y_ is smiling and the man part of _y_ is frowning]],
    ),
    (
      "IP2",
      [$lambda$_x_ : _x_ has a unique maximal woman part \
        and a unique maximal man part. \ the woman part of x is smiling and \
        the man part of x is frowning],
    ),
    (
      "QP1",
      [$lambda$_Q_$forall$_y_[_y_ is woman+man \
        $and$ _y_ came in together] $arrow$ _Q(y)_],
    ),
    (
      "IP3",
      [the woman part of g(2) is smiling \ and the man part of g(2) is frowning],
    ),
    (
      "DP†1",
      [the woman part \ of g(2)],
    ),
    (
      "DP‡1",
      [the man part \ of g(2)],
    ),
  ),
  dominance: (
    (from: "muP4", to: "muP1", ctrl: (-6.1, 8.5)),
    (from: "muP4", to: "muP2", ctrl: (-6, 5)),
  ),
  scale: 0.8,
  spread: 0.8,
  terminal-branch: true,
)
```

Code 11: Code for Tree 10



Tree 10: Complex tree with multi-line annotation

5.1. Systems of spheres

Not every semantic representation is a tree. For conditional and counterfactual semantics in the tradition of Lewis (1973), propositions are drawn against a system of nested spheres of possible worlds centred on the world of evaluation. The `#spheres()` function draws exactly that: a filled solid sphere, four dotted concentric guides around it, and any number of labelled parabolic boundaries cutting across them, as shown in Diagram 1.

The five concentric lines are numbered from the centre outwards, which gives us the `depth` scale used throughout the function. Depth `3` is the boundary of the solid sphere itself, depths `1` and `2` lie inside it, and depths `4` and `5` lie outside it. Depth `0` refers to the centre of the diagram. The world of evaluation is set with `center-label`, which defaults to w.

Each entry in `parabolas` is a dictionary. Only `label`, `angle`, and `depth` are required: `angle` places the parabola around the diagram in degrees, where `0` opens to the right and positive values rotate counter-clockwise, and `depth` selects the circle its vertex sits on. The optional `touch-angle` controls where on that circle the vertex is placed, and defaults to the parabola's own angle. Because labels sit at the end of a curve rather than on a node, `label-position` and `label-offset` are available for the inevitable adjustments. Adding `shade: (inner, outer)` fills the region of that parabola lying between two circle depths, which is how Diagram 1 marks φ^+ and φ without any separate `shading` entry. For regions defined by more than one parabola, the `shading` parameter takes entries with `inside`, `outside`, `band`, and `fill`: named inside regions are intersected, outside regions are subtracted, and the result is clipped to the radial band given by `band`. Naming a parabola with `id` is what makes it referable there.

All parabolas share a single `parabola-opening`, and both tails of a parabola are drawn to the same extent, computed so that the endpoints clear the outermost guide. `parabola-tail-gap` pushes that virtual boundary further out when longer tails are wanted. The remaining arguments are cosmetic: `scale`, `circle-fill`, `shading-fill`, `stroke`, `guide-stroke`, and `line-width`. Table 3 lists them all.

```
#spheres(
  center-label: [w],
  parabolas: (
    (
      label: $phi^+$,
      angle: 38,
      touch-angle: 45,
      depth: 4,
      label-offset: (0.1, 0.30),
      shade: (4, 5),
    ),
    (
      label: $phi$,
      angle: 41,
      touch-angle: 45,
      depth: 2,
      label-offset: (0.68, 0.23),
      shade: (2, 3),
    ),
    (
      label: $psi$,
      angle: 328,
      touch-angle: 86,
      depth: 3,
      label-position: -1.0,
      label-offset: (0.08, -0.04),
    ),
  ),
)
```

Code 12: Code for Diagram 1

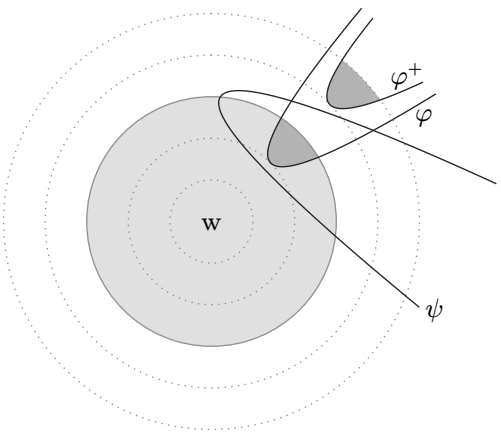


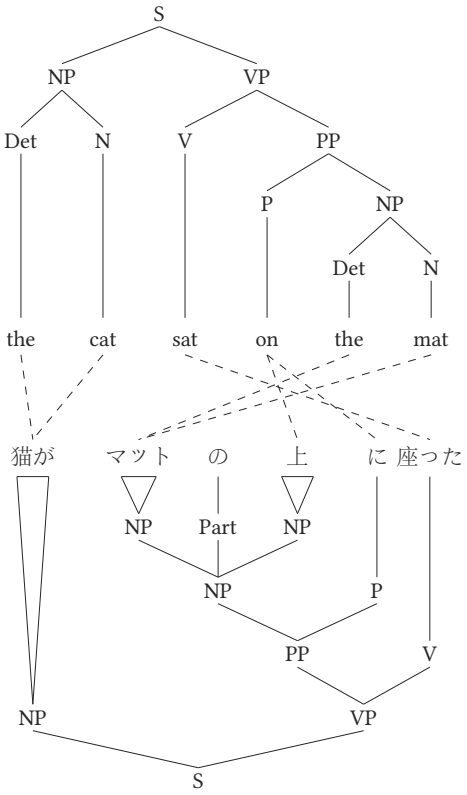
Diagram 1: A Lewis-style system of spheres

6. Bilingual trees

In one of the examples found in David Chiang's tutorial on `tikz-qtree` (Chiang, 2009), English and Japanese trees are compared in a single figure. This is accomplished here with the function `#garden()`, which allows for multiple trees to be built on top of each other. Because two is probably the ideal number of trees for this use case, that is the scenario illustrated in Tree 11. Here, again, the tree number is referenced by a suffix. For example, if we want to link the second N in tree 1 to the third NP in tree 2, we add `("n2-1", "np3-2")`. This is relatively intuitive, and automatic labels allow for Code 13 to be considerably more minimal than what we would need in LaTeX. Finally, to have a "mirror image" of two trees, one of the trees must be upside-down. This is why the `direction` parameter exists, discussed further in Section 10.4. We simply specify `direction: "up"` to flip a tree vertically, as shown in Tree 11. The Japanese verb has been corrected from the original tutorial's apparent `土` typo to `座った`. As in `#tree()`, `show-refs: true` prints every reference name. In `#garden()`, bare names target leaves and triangle content, and `-top` names target node labels.

```
#garden(
  ( // tree 1
    input: "[S [NP [Det the] [N cat]] [VP [V sat] [PP [P on] [NP [Det the] [N mat]]]]]",
    spread: 1.55, // This ensures that the trees' width is virtually the same
    content-size: 1
  ),
  ( // tree 2
  input: "[S [NP 猫が] [VP [PP [NP [NP マット] [Part の] [NP 上] ] [P に]] [V 座った]]]",
  direction: "up",
  content-size: 1
  ),
    // Inter-tree lines (dashed)
  equivalence: (
    ("Det1-1", "NP1-2"), ("P1-1", "P1-2"), ("P1-1", "NP4-2"),
    ("N1-1", "NP1-2"), ("N2-1", "NP3-2"), ("Det2-1", "NP3-2"),
    ("V1-1", "V1-2"),
  ),
  gap: 2.5,
  scale: 0.7
)
```

Code 13: Code for Tree 11



Tree 11: Bilingual trees — adapted from Chiang's tutorial on `tikz-qtree`

```
\begin{tikzpicture}
   \begin{scope}[frontier/.style={distance from root=150pt}]
   \Tree [.S [.NP [.Det \node(e1){the}; ]
             [.N \node(e2){cat}; ] ]
           [.VP [.V \node(e3){sat}; ]
             [.PP [.P \node(e4){on}; ]
               [.NP [.Det \node(e5){the}; ]
                 [.N \node(e6){mat}; ] ] ] ] ]
   \end{scope}
   \begin{scope}[xshift=9pt,yshift=-5in,grow'=up,
             frontier/.style={distance from root=150pt}]
   \tikzset{every leaf node/.style={font=\ja}}
   \Tree [.S [.NP \node(j1){猫が}; ]
           [.VP [.PP [.NP [.NP \node(j2){マット}; ]
                     [.Part \node(j3){の}; ]
                     [.NP \node(j4){上}; ] ]
                 [.P \node(j5){に}; ] ]
             [.V \node(j6){座った}; ] ] ]
   \end{scope}
   \begin{scope}[dashed]
      \draw (e1)--(j1);
      \draw (e2)--(j1);
      \draw (e3)--(j6);
      \draw (e4)--(j4);
      \draw (e4)--(j5);
      \draw (e5)--(j2);
      \draw (e6)--(j2);
   \end{scope}
\end{tikzpicture}
```

Code 14: Equivalent code in LaTeX to produce Tree 11

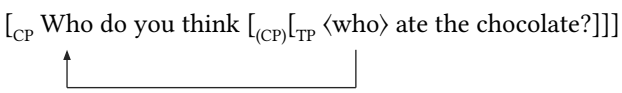
By default, stacked trees will use `bottom: true` to ensure that alignment is optimal — see Tree 18 in Section 10.1. Consequently, terminal branches are displayed to avoid empty space between nodes and their content.

7. In-line movement

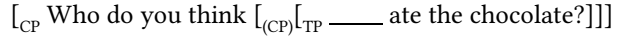
In-line representations also make use of automatic labelling. Every word we add to a sentence is its own label. As a result, we can easily draw an arrow from `who2` to `who1` in Example 1. Because the arrows extend below the text baseline, they may overlap with subsequent content. Setting `protect: true` reserves vertical space below the output to prevent this. By default, `protect` is `false`, which is the correct setting when `#move()` is placed inside a table cell (e.g., in numbered examples with `#eg()`), where extra height would misalign adjacent cells — see Section 8 below. Because these representations are almost always used in the context of a numbered example, `protect: false` is favoured.

The examples below demonstrate how to change line styles for arrows and how to delink a given arrow using its position. For example, if `delinks: (0,)`, a delink is added to the first movement arrow added to the code — see last two examples in Example 1.

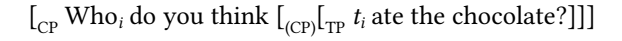
```
#move("[CP Who do you think [(CP)[TP<who>ate the chocolate?]]]",
  arrows: ((from: "who2", to: "who1"),),
  protect: true,)
```



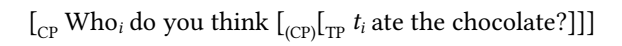

```
#move("[CP Who do you think [(CP)[TP __ ate the chocolate?]]]",
  arrows: ((from: "t1", to: "who1", dash: "wavy", color: red),),
  protect: true,)
```



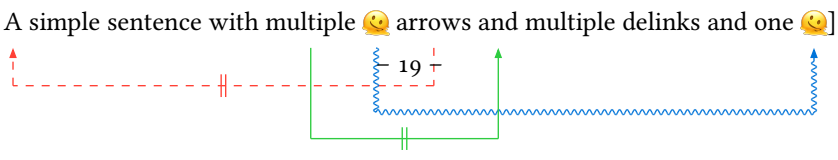

```
#move("[CP Who do you think [(CP)[TP t_i ate the chocolate?]]]",
  arrows: ((from: "t1", to: "who1", dash: "dashed"),),
  protect: true,)
```




```
#move("[CP Who do you think [(CP)[TP t_i ate the chocolate?]]]",
  arrows: ((from: "t1", to: "who1", dash: "dotted"),),
  delinks: (0,), // delink the first (0) arrow
  protect: true,)
```




```
#move("[S A simple sentence with multiple 🫠 arrows and multiple delinks and one 🫠]",
  arrows: (
    (from: "simple1", to: "show1", dash: "dotted", color: black),
    (from: "arrows1", to: "a1", dash: "dashed", color: red),
    (from: "🫠1", to: "🫠2", dash: "wavy", color: blue),
    (from: "multiple1", to: "and1", dash: "solid", color: green),
  ),
  delinks: (0, 2),
  protect: true,)
```



8. Numbered examples

The examples shown in Example 1 are usually placed in numbered examples. This can be accomplished with the function `#eg()`, which is the same implementation found in **phonokit**, to keep both packages consistent.

> **Important.** To use this function, add this line after importing the package: `#show: eg-rules`

The simplest use of `#eg()` is to wrap content directly. The example number is generated automatically:

```
#eg[#move(
  "[CP Who do you think [(CP)[TP<who>ate the chocolate?]]]",
  arrows: ((from: "who2", to: "who1"),),
)] <eg-single>
```

Code 15: Single-item numbered example

(1) [$_{CP}$ Who do you think [$_{(CP)}$[$_{TP}$ ⟨who⟩ ate the chocolate?]]]



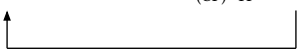
For sub-examples, use the list syntax (`-`). Each item is automatically lettered (`a.`, `b.`, etc.). The `labels` argument allows individual sub-examples to be referenced, as in (2a) and (2b). The example as a whole can also be referenced, as in (2).

```
#eg(labels: (<s-plain>, <s-move>))[
  - Who do you think saw Mary?
  - #move(
      "[CP Who do you think [(CP)[TP<who>saw Mary]]]",
      arrows: ((from: "who2", to: "who1", dash: "solid", color: black),),
    )
] <eg-wh>
```

Code 16: Sub-examples with labels

(2) a. Who do you think saw Mary?

b. [$_{CP}$ Who do you think [$_{(CP)}$[$_{TP}$ ⟨who⟩ saw Mary]]]



Without `labels`, sub-examples are simply not "referenceable" individually — only the example as a whole can be labelled and referenced. The `title` argument places a title on the same line as the example number, with the content below. Finally, the `caption` argument is only used if you plan to have a table of contents exclusively for examples, which covers both `#eg()` and `#gloss()` (discussed next). In (3), a `caption` has been added, but nothing is printed. In Section 9, an outline is produced where this caption will be useful.

```
#eg(title: [Wh-movement in English], caption: [Wh-movement in English])[
  - Who do you think saw Mary?
  - #move(
      "[CP Who do you think [(CP)[TP<who>saw Mary]]]",
      arrows: ((from: "who2", to: "who1"),),
    )
] <eg-title>
```

Code 17: Example with `title` to generate (3)

(3) Wh-movement in English

a. Who do you think saw Mary?

b. $[_{CP}$ Who do you think $[_{(CP)}[_{TP}$ ⟨who⟩ saw Mary]]]



A note on figure spacing. This applies to **phono*kit*** as well as to **syn*kit***: If your document uses a custom `#show figure` rule to add vertical spacing around figures, you may notice misalignment in sub-example labels. This happens because `#subex-label()` uses figures internally, and added spacing will disrupt table cell alignment. To fix this, exclude `linguistic-subexample` from your spacing rule. For example, a document that adds `1em` vertical spacing before and after each figure should have something along these lines:

```
#show figure: it => {
  if it.kind == "linguistic-subexample" {
    it
  } else {
    v(1em)
    it
    v(1em)
  }
}
```

Code 18: Excluding linguistic examples from custom figure spacing specifications

9. Glosses

The `#gloss()` function is meant to be a lightweight companion to `#eg()`, especially when examples need to share numbering across **syn*kit*** and **phono*kit***. For a more complete glossing solution, I strongly recommend using the `eggs` package (Shaldov, 2025), which is a dedicated package.

For numbered examples involving glosses, where alignment is essential, you can use the `#gloss()` function. Its implementation is simple (similar to `#eg()` discussed above): it uses regular expressions to look for spaces, which trigger a split that guarantees alignment across words. Its syntax is also minimal, since it's simply a list. Curly braces can be used to trigger small caps. Both functions (`#gloss()` and `#eg()`) share the same numbering, as expected, since both are numbered examples. See Code 19, which generates (4) (note that a `caption` is added here, which will be relevant later in this section).

```
#gloss(spacing: 1.2em, caption: [An example from Portuguese])[
  - eu gosto de maçã
  - I like-{1sg.prs} of apple
  - 'I like apples.'
] <eg-gloss-1>
```

Code 19: Code to generate (4)

(4) eu gosto de maçã
I like-1SG.PRS of apple
'I like apples.'

Labels such as `<eg-gloss-1>` seen in Code 19 are global, but you can also add labels for subglosses (exactly like `#eg()`) with the `labels` argument. Likewise, a `per` argument exists in case you want to have multiple glosses in the same example with a custom number of lines for whatever reason. The reason for `per` is flexibility: if you wanted to have glosses with 5 lines each (line 5 being the translation), you would simply define `per: 5`. Code 20 shows how (5) is generated. Given that `labels` have been added in Code 20, we can now refer to (5a) and (5b).

```
#gloss(per: 3, labels: (<eg-sub-pt1>, <eg-sub-pt2>))[
  - eu gosto de maçã
  - I like.{1sg.prs} of apple
  - 'I like apples.'
  - elle aime les pommes
  - she like.{3sg.prs} {def.pl} apple.{pl}
  - 'She likes apples.'
] <eg-sub-pt>
```

Code 20: Code to generate (5)

(5) a. eu gosto de maçã
I like.1SG.PRS of apple
'I like apples.'

b. elle aime les pommes
she like.3SG.PRS DEF.PL apple.PL
'She likes apples.'

Let's explore some additional cases. In (6), we see an example adapted from (O'Grady & Archibald, 2017, p. 275).[1] This is a situation where we may want to have four lines for a gloss, where a given line simply shows an orthographic form that should not be parsed by the `#gloss()` function. That's why the `escape` argument exists. In (6), the first line (number `0`) is "free" from the parsing used elsewhere in the example.

[1] The negation suffix has been updated from the source's `-ngit-` to the modern standard geminate `-nngit-`. Other features of the original spelling — e.g., the clusters `gss` and `tl`, and the final `-puq` — reflect a pre-reform orthographic tradition (ultimately traceable to older West Greenlandic sources) and have been retained here as in the cited source.

```
#gloss(per: 4, escape: (0,), caption: [An example from Inuktitut])[
  - Qasuiirsarvigssarsinngitluinarnarpuq
  - Qasu -iir -sar -vig -ssar -si -nngit-luinar -nar -puq
  - tired not cause-to-be place-for suitable find not-completely someone 3.{sg}
  - 'Someone did not find a completely suitable resting place.'
] <gloss-inuktitut>
```

Code 21: Code to generate (6)

(6) Qasuiirsarvigssarsinngitluinarnarpuq
Qasu -iir -sar -vig -ssar -si -nngit-luinar -nar -puq
tired not cause-to-be place-for suitable find not-completely someone 3.SG
'Someone did not find a completely suitable resting place.'

Here's another case, this time from Turkish (O'Grady & Archibald, 2017), where we have IPA symbols[2] and only two lines (`per: 2`). This is shown in (7).

```
#gloss(per: 2)[
  - [kœj]
  - 'village'
] <gloss-turkish>
```

Code 22: Code to generate (7)

(7) [kœj]
'village'

Finally, a case with a title, also from Turkish (O'Grady & Archibald, 2017), is shown in (8).

```
#gloss(title: [SOV (Turkish)], caption: [Another example])[
  - Hasan œkyz-y al-dɨ
  - Hasan ox-{acc} buy-{pst}
  - 'Hasan bought the ox.'
] <gloss-turkish2>
```

Code 23: Code to generate (8)

(8) SOV (Turkish)
Hasan œkyz-y al-dɨ
Hasan ox-ACC buy-PST
'Hasan bought the ox.'

[2]Because Typst has native UTF-8 support, you can simply add whatever symbol you want.

We can now generate a simple table of contents that targets only the examples for which we set a `caption` above. These include both `#eg()` and `#gloss()`. If you don't add a `caption` for an example, it will not appear in the table of contents generated below (see Code 24).

```
#outline(
  title: [List of Examples],
  target: figure.where(kind: "linguistic-example"),
)
```

Code 24: Code to create an outline for examples

List of Examples

(3) Wh-movement in English 21
(4) An example from Portuguese 22
(6) An example from Inuktitut 23
(8) Another example 23

If you also use **phonokit** in your document, the outline above will integrate phonology and syntax examples seamlessly, even though both packages use two different functions, namely `#ex()` and `#eg()`.

10. Extras

This section introduces some important parameters to further adjust syntax trees.

10.1. Spacing

Table 1 lists key parameters to understand how spacing works in `#tree()`.

Parameter	Purpose
`spread`	Global spacing between sister nodes (horizontal)
`spread-local`	Local spacing between sister nodes (based on labels)
`drop`	Global spacing between levels (vertical)
`drop-local`	Local spacing between levels (based on labels)

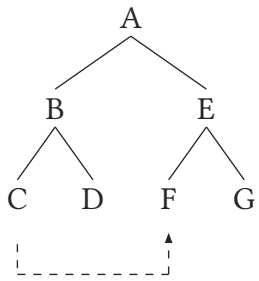
Table 1: Spacing parameters in `#tree()`

To see these parameters in action, let's examine Tree 12 to Tree 17. These are all based on the base code shown in Code 25. Tree 12 shows what a typical tree looks like without any custom specification for the parameters in Table 1. An arrow is added to all trees to show the user how they adjust automatically as we adjust any spacing parameter (since they're based on labels and not on absolute coordinates).

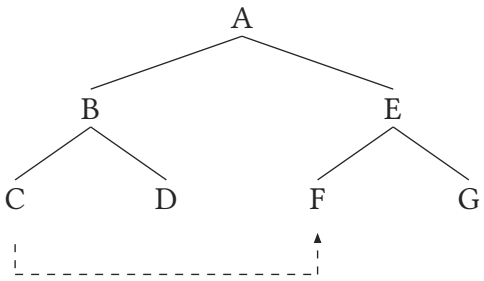
If we want the tree as a whole to have more horizontal separation, `spread` can be used. Tree 13, for example, increases the spread by 100% (from `1` to `2`). But we often want to adjust only one sister-to-sister interval; that's why `spread-local` exists, as shown in Tree 14, which reduces the gap after C (that is, the C-D spacing). The parameter `drop` works on the same direct scale for vertical spacing, as can be seen in Tree 15 to Tree 17.

```
#tree("[ A [ B [C] [D] ] [ E [F] [G] ] ]", arrows: ("C1", "F1"), .. .. ..)
```

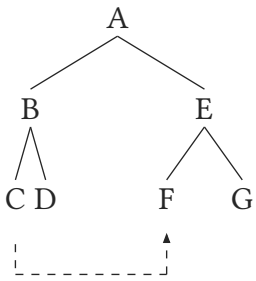
Code 25: Code for Tree 12 through Tree 17



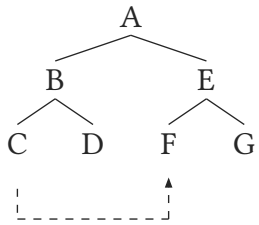
Tree 12: Default



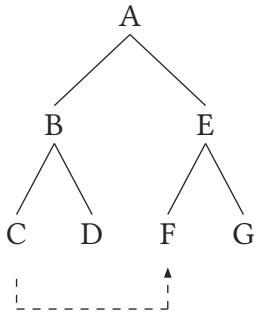
Tree 13: `spread: 2`



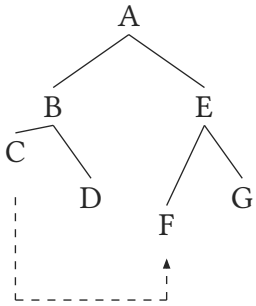
Tree 14: `spread-local` (C-D)



Tree 15: `drop: 0.7`



Tree 16: `drop: 1.2`

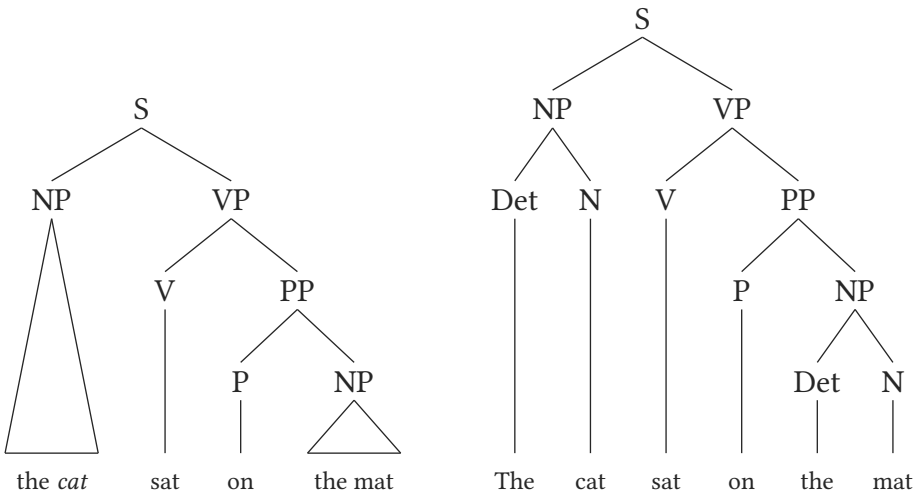


Tree 17: `drop-local` (C-F)

Another possible adjustment to spacing involves the vertical alignment of node contents, i.e., the spacing between a node and its content. The parameter `bottom: true` will pull the tree's content down and bottom-align it. This will trigger terminal branches by default, as already mentioned.

```
#tree(
  "[S [NP the *cat*] [VP[V sat][PP[P on] [NP the mat]]]]",
  bottom: true,
)
#tree(
  "[S [NP [Det The] [N cat] ] [VP[V sat][PP[P on] [NP [Det the] [N mat]]]]]",
  bottom: true,
)
```

Code 26: Code for Tree 18



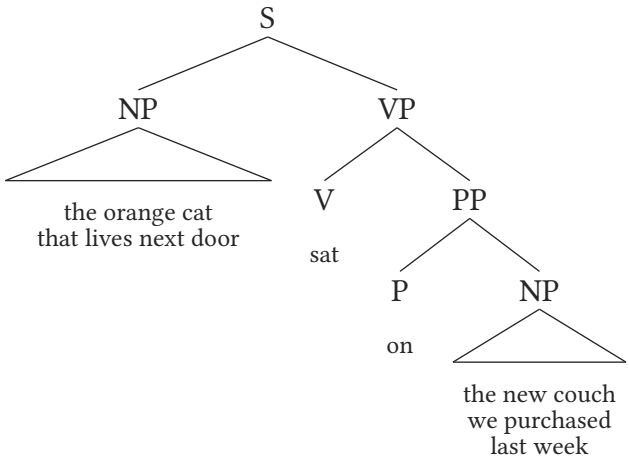
Tree 18: Bottom alignment — looks better without triangles

10.2. Line breaks

Breaking lines is easy with `\\n`. This avoids having phrases that are too wide.

```
#tree(
    "[S [NP the orange cat \\n that lives next door]
      [VP[V sat][PP[P on]
      [NP the new couch \\n we purchased \\n last week]]]]",
  )
```

Code 27: Code for Tree 19



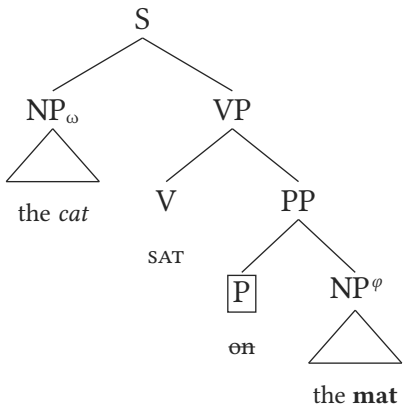
Tree 19: Breaking lines within phrases

10.3. Formatting

Let's now explore some important consequences of using a string as our main input. What if we want to use bold, italics, small caps, Greek letters, etc.? If you have used Typst before, you know that you can simply add whatever symbol you need directly into the document (UTF-8 is natively accepted). But `#tree()` also has some presets to make this easier. Tree 20 shows how to adjust the formatting of strings in the tree. Greek letters are interpretable as long as you use `\\`. For italics, you use `*cat*`; for bold, `**mat**`, for small caps, `@sat@`. You can also strike through text with `~` (see Tree 20). Finally, subscripts and superscripts can be easily added with `_` and `^`, respectively. An additional parameter, `highlight`, exists for drawing a rectangle around any given node (equivalent to `\node[draw]` in `tikz-qtree`) — we refer to `P` as `"P1"` in Code 28 ("first [and only] P from the top of the tree").

```
#tree("[S [NP_\\omega the *cat*] [VP[V @sat@][PP[P ~on~] [NP^\\phi the **mat**]]]]",
            highlight: ("P1",))
```

Code 28: Code for Tree 20



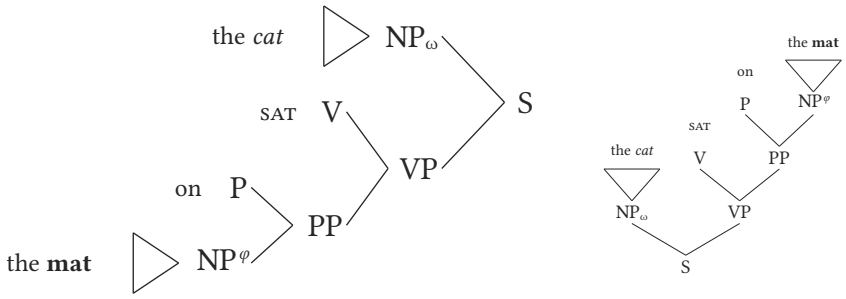
Tree 20: The basics: highlight, bold, italics, small caps, super- and subscripts

10.4. Direction

Trees can also change direction with the `direction` parameter. By default, `direction: "down"`, but you can set it to `"up"`, `"left"`, or `"right"`, as shown in Code 29. In addition, you can use the `scale` parameter to increase or decrease the size of your tree. In Tree 21, the tree on the left uses `scale: 1` (default), while the tree on the right uses `scale: 0.6` (60% of its original size).

```
#tree("[S [NP_\\omega the *cat*] [VP[V @sat@][PP[P on] [NP^\\phi the **mat**]]]]",
            direction: "left")
```

Code 29: Code for Tree 21



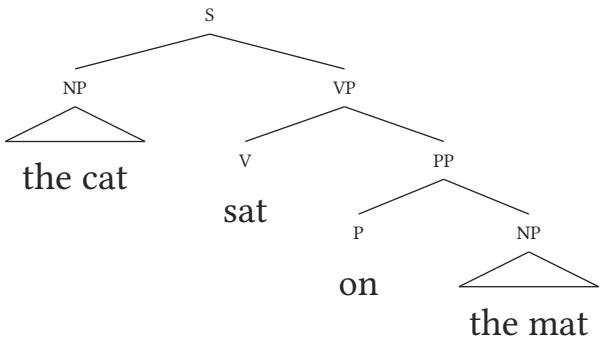
Tree 21: Changing tree direction and size

10.5. Sizing

Sizing and spacing are two key parameters in any tree. By default, the content of each node has 80% of the size of the node itself, but this can be changed with the `content-size` parameter (defaults to `0.8`). Tree 22 uses `content-size: 1.2` to make the content larger than the nodes. In Code 30, you will also find two other parameters: `drop` and `spread`. These help you set vertical and horizontal spacing for the tree. If you wish to change the size of nodes, `node-size` is also available.

```
#tree(
  "[S [NP the cat] [VP[V sat][PP[P on] [NP the mat]]]]",
  content-size: 1.2,
  node-size: 0.6,
  drop: 0.8,
  spread: 1.5,
)
```

Code 30: Code for Tree 22



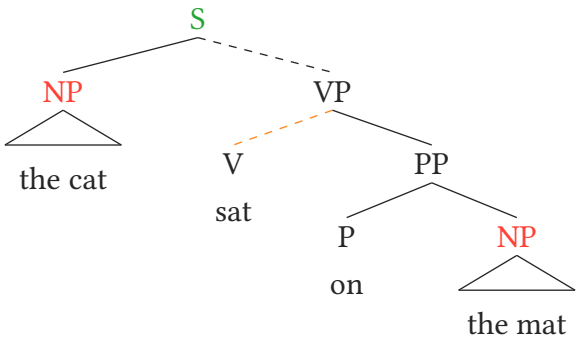
Tree 22: Size and spacing: larger content, smaller nodes

10.6. Branch styling

Different parameters exist for styling a tree, most notably `color` and `dash-branches`. Tree 23 uses both parameters. The parameter `color` is flexible and can refer to both branches or text, depending on the parameters passed to it. For example, `color: ("S1", green)` will target a node (S), while `color: ("VP1", "V1")` will target *the relationship* between nodes, i.e., a branch.

```
#tree(
  "[S [NP the cat] [VP[V sat][PP[P on] [NP the mat]]]]",
  content-size: 1,
  drop: 0.8,
  spread: 1.5,
  dash-branches: (
    ("VP1", "V1"),
    ("S1", "VP1"),
  ),
  color: (
    ("S1", green.darken(20%)),
    ("NP1", red),
    ("NP2", red),
    ("P1down", blue),
    ("VP1", "V1", orange),
  ),
)
```

Code 31: Code for Tree 23



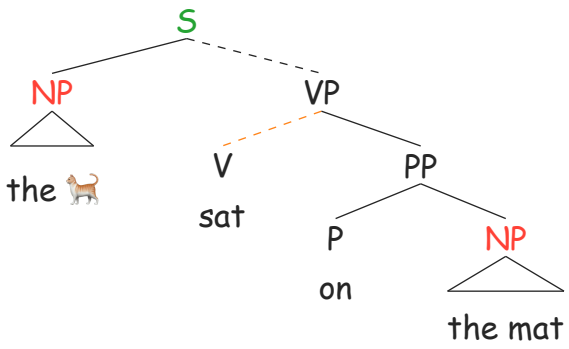
Tree 23: A colourful tree

10.7. Font

While **synkit** inherits the document font by default, you can always choose whatever font you prefer for each function. Tree 24 illustrates this by setting `font: "Comic Sans MS"`.

```
#tree(
  "[S [NP the 🐈] [VP[V sat][PP[P on] [NP the mat]]]]",
  content-size: 1,
  drop: 0.8,
  spread: 1.5,
  dash-branches: (
    ("VP1", "V1"),
    ("S1", "VP1"),
  ),
  color: (
    ("S1", green.darken(20%)),
    ("NP1", red),
    ("NP2", red),
    ("P1down", blue),
    ("VP1", "V1", orange),
  ),
  font: "Comic Sans MS",
)
```

Code 32: Code for Tree 24



Tree 24: Changing the font locally (emojis natively supported)

11. Future work and acknowledgments

The package is in its infancy, so there will be bugs and limitations. That is why comments and suggestions are especially welcome, as they will speed up the process of improving precision and coverage. Simply open an issue in the package's repo. I am not a syntactician, so I've benefitted greatly from the help of my colleague Vincent Rouillard.

References

Carnie, A. (2013). *Syntax: A Generative Introduction* (3rd ed.). Blackwell Publishing.

Chiang, D. (2009). *tikz-qtree – Use existing qtree syntax for trees in TikZ*. https://ctan.org/pkg/tikz-qtree

Dreamin, M., & Varner, N. (2024). *Tinymist: An integrated language service for Typst*. https://github.com/Myriad-Dreamin/tinymist

Elliott, P. D., Nicolae, A. C., & Sauerland, U. (2022). Who and what do who and what range over cross-linguistically?. *Journal of Semantics*, *39*(3), 551–579.

Fejfar, J., & Funke, M. (2024). *CeTZ: A library for drawing with Typst*. https://github.com/cetz-package/cetz

Fox, D., & Johnson, K. (2016). QR is restrictor sharing. In K.-min Kim, P. Umbal, T. Block, Q. Chan, T. Cheng, K. Finney, M. Katz, S. Nickel-Thompson, & L. Shorten (Eds.), *Proceedings of the 33rd West Coast Conference on Formal Linguistics: Proceedings of the 33rd West Coast Conference on Formal Linguistics*.

Lewis, D. (1973). *Counterfactuals*. Blackwell.

O'Grady, W., & Archibald, J. (2017). *Contemporary Linguistic Analysis: An Introduction* (7th ed.). Toronto: Pearson.

Shaldov, A. (2025). *eggs: linguistic examples with minimalist syntax*. https://github.com/retroflexivity/typst-eggs

Živanović, S. (2017). *The forest package: Drawing linguistic (and other) trees*. https://ctan.org/pkg/forest

Appendix

A. Argument reference

Table 2 lists all available parameters in `#tree()`.

Argument	Type	Default	Description
`arrows`	array	`()`	Cross-node arrows. Each entry is `(from, to)` or a dict with keys `from`, `to`, `color`, `bend`, `shift`, `dash`, `line-width`
`scale`	number	`1.0`	Uniform scale factor for the whole tree
`triangle`	array	`()`	Anchor names whose branch renders as a triangle for elided structure
`content-size`	number	`0.8`	Size multiplier for leaf content text
`node-size`	number	`1.0`	Size multiplier for node label text
`curved`	bool	`false`	Draw arrows as Bézier curves instead of rectangular paths
`direction`	string	`"down"`	Growth direction: `"down"`, `"up"`, `"right"`, or `"left"`
`highlight`	array	`()`	Anchor names to draw a box around. Use `"DP1-down"` to box the leaf content
`bottom`	bool	`false`	Align all leaves at the lowest level regardless of depth
`terminal-branch`	bool	`false`	Draw branches from non-terminal nodes to terminal (leaf) nodes
`dash-branches`	array	`()`	Array of `(parent, child)` anchor pairs whose branch is dashed
`delinks`	array	`()`	Anchor names where a delink mark (two bars) is drawn on the arrow shaft
`index`	array	`()`	Coreference subscripts. Array of single-key dicts, e.g. `(("CP1": "i"),)`
`append`	array	`()`	Extra subscript text appended after a node label, e.g. `(("VP1", "0"),)`
`drop`	number	`1.0`	Multiplier for vertical distance between levels
`drop-local`	array	`()`	Per-level or per-node branch length multipliers, e.g. `(1, 1.5)` or `("IP2", 0.5)`
`spread`	number	`1.0`	Horizontal width per leaf in canvas units
`spread-local`	array	`()`	Per-level or per-node horizontal gap multipliers between sisters
`dominance`	array	`()`	Long-distance dominance lines. Entries: `("from", "to")` or dict with `ctrl`, `color`, `dash`
`color`	array	`()`	Colorize nodes, leaves, or branches. E.g. `("NP1", red)` or `("VP1", "V1", yellow)`
`annotation`	array	`()`	Semantic annotations between node label and branches, e.g. `(("DP1", $lambda$),)`
`annotation-size`	number	`0.70`	Size multiplier for annotation text
`annotation-leading`	length or `auto`	`auto`	Line spacing for multi-line annotations
`numbers`	array	`()`	Circled numbers placed to the left of node labels, e.g. `(("DP1", 2),)`
`numbers-size`	number	`0.85`	Size multiplier for circled numbers relative to node labels
`line-width`	number	`1.0`	Stroke width multiplier for all tree lines
`font`	string	`none`	Font family used in the function (local scope)

Table 2: All arguments in `#tree()`

Table 3 lists all available parameters in `#spheres()`, described in Section 5.1.

Argument	Type	Default	Description
`center-label`	content	`$w$`	Label for the world of evaluation at the centre
`parabolas`	array	`()`	Proposition boundaries. Each entry is a dict with `label`, `angle`, `depth`, and optionally `id`, `touch-angle`, `label-position`, `label-offset`, `shade`
`shading`	array	`()`	Regions built from named parabolas. Keys are `inside`, `outside`, `band`, `fill`
`parabola-opening`	number	`4.0`	Shared opening width of every parabola
`parabola-tail-gap`	number	`1.0`	Extra distance past the outermost guide that tails extend to
`scale`	number	`1.0`	Uniform scale factor for the whole diagram
`circle-fill`	color	`luma(88%)`	Fill of the solid sphere
`shading-fill`	color	`luma(70%)`	Default fill for shaded regions
`stroke`	color	`luma(15%)`	Colour of the sphere outline and parabolas
`guide-stroke`	color	`luma(58%)`	Colour of the four dotted guide circles
`line-width`	number	`1.0`	Line width multiplier

Table 3: All arguments in `#spheres()`

B. How can I use Typst offline?

This vignette assumes that you know about Typst, but you may not be very familiar with it. That's why this section exists. For example, while the online app at Typst.app is very useful and practical, most of us prefer to work offline. **How can you use Typst offline then?**

One of the best IDE options out there is to use VS Code with the extension Tinymist (Dreamin & Varner, 2024) — the extension is therefore available for Positron, which is the successor to RStudio. Tinymist is also available as a plugin for NeoVim users. All these options work extremely well because Tinymist is great, and I haven't had any issues thus far: compilation is instantaneous, and `bib` files also work flawlessly.[3]

C. How do packages work in Typst?

If you've used R, Python, LaTeX, etc., you are used to installing packages and then importing them. This vignette has imported **syn*kit***, of course, which in turn imports CeTZ (Fejfar & Funke, 2024) as a dependency. As you start using Typst, you will notice that it works a bit differently, and this may not be self-evident at first. As seen in Section 1.1, there are basically two ways to load and use a package, both of which require the function `#import` inside your `typ` document — notice that you don't install a package *per se*. The traditional way is to import a package from the official Typst collection/repository, which means adding `#import "@preview/synkit:0.2.1": *` to your `typ` document if you plan on using **syn*kit*** (assuming version `0.2.1`). The `@preview` bit indicates that the package comes from Typst's official repository. This is what you should do most of the time. Typst packages are cached once you compile a document with a given package.

Another option is to fork, clone or download a package from GitHub and import its `lib.typ` file instead: `#import "PACKAGE_DIRECTORY/lib.typ": *`. There's only one caveat: Typst restricts imports to

[3] Provided that they are clean and do not have any problems regarding fields, repeated entries, etc.

files within the compilation root and its subdirectories (i.e., you can't load `lib.typ` if the package is in a parent directory or elsewhere in your system). Thus, you may need to use symlinks (this is the same strategy applied to `bib` files if you don't want to have a local copy of your references).

```
# From your working directory:
ln -s PATH_TO_PACKAGE_DIRECTORY package_name
```

Code 33: Creating a symlink to local package

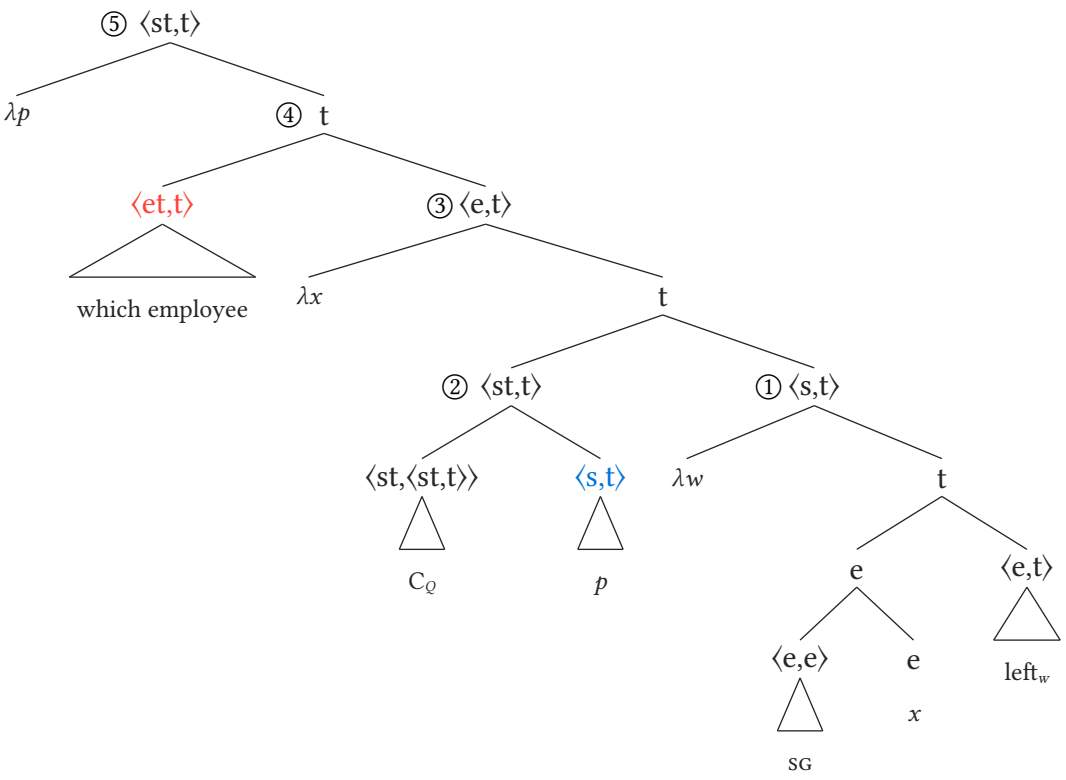
Finally, Typst's repository contains sub-directories to keep track of each version of a given package. When a package is updated, nothing happens to the existing version of the package. Instead, a new directory is added with the updated version. That's why you specify the *version* of a package upon importing: `#import "@preview/synkit:0.2.1": *`. If you go to Typst's repository on GitHub, you will see that the repository for **synkit** has one sub-directory for each version of the package. This means that you always know which version of a package you are using. And because previous versions are not removed, backwards compatibility is not an issue. If you are used to LaTeX and you have the habit of frequently updating your packages, you will appreciate this, as there's no risk of recompiling your document and running into errors because one of the packages you use has been updated with breaking changes.

D. Additional trees

Example from Elliott et al. (2022, p. 10) showing the flexibility of labels and the `numbers` argument.

```
#tree(
  "[<st,t> [\\lambda*p*] [t [<et,t> which employee] [<e,t> [\\lambda*x*]
      [t [<st,t> [<st,<st,t>> C_Q] [<s,t> *p* ] ] [<s,t> [\\lambda*w*]
          [t [e [<e,e> @sg@ ] [e *x*] ] [<e,t> left_w] ] ] ] ] ] ]",
    color: (
      ("ett1", red),
      ("st1", blue),
    ),
    numbers: ( // Label stripping ignores <, >, and , in node names
      ("stt1", 5),
      ("t1", 4),
      ("et1", 3),
      ("stt2", 2),
      ("st2", 1),
    ),
    numbers-size: 1.2,
)
```

Code 34: Code for Tree 25



Tree 25: Numbers and flexible labels (example with colors)